AI Changes the Delivery System and the Product Itself

Vitalii Oborskyi

AI changes how teams deliver software and what they must control when model judgment enters a running product. A practical explanation of flow, evidence, authority and recovery.

---

1. Delivery speed depends on the whole flow
2. Model judgment changes the product's responsibilities
3. Evidence must fit the decision
4. Control must work before and after consequence
5. Leadership connects the decisions
6. Further reading and the talk

AI enters software in two different places. A developer can use it to produce code, tests or documentation. A running product can use a model to interpret a request, choose a response or propose an action. The first changes how we deliver software. The second changes the responsibilities of the system we deliver.

These changes can occur independently. AI-written code can implement an ordinary, explicitly specified application. A human-written application can depend on model judgment at runtime. When both changes occur together, leaders need to examine the delivery flow and the product's behavior separately, then connect their decisions.

Delivery speed depends on the whole flow

Writing code is one part of delivery. Requirements, design, review, testing, integration, deployment and operation still consume capacity. If generation accelerates while review or integration remains constrained, faster production of changes may create a larger queue rather than more useful releases. If coding was the constraint, its acceleration may move that constraint elsewhere.

This is a conditional mechanism, not a prediction that AI makes every team slower. A team may gain throughput, reduce effort or free capacity without creating overload. AI can also help with review, testing and understanding. The question is what happens to the complete flow in this team, for comparable work, under its actual operating conditions.

That requires more than counting generated lines. Look at review age, work in progress, batch size and lead time alongside delivered value, rework and production stability. Investigate changes in duplication, complexity and dependencies in context; none is an automatic verdict on code quality. Include the people who review and maintain the system: their workload, ownership coverage and ability to recover it matter.

Human comprehension can become a constraint when a team accepts code faster than it can explain, verify and maintain it. The PMDay talk presents that gap as an illustrative hypothesis. It does not establish a universal human limit or a measured curve. A useful practical check is whether the team can diagnose and restore an important component when its AI tool cannot produce a satisfactory repair. Collective capability matters; every person need not memorize every line.

Small batches, intake limits, stronger specifications, living documentation and AI-assisted review are candidates to test. Choose a baseline, bounded trial and decision point. Check whether the practice improves the flow without shifting cost into rework, on-call load or loss of understanding. Another model's review is additional evidence, with its own failure modes.

Model judgment changes the product's responsibilities

The second change begins when a consequential runtime responsibility depends partly on probabilistic model judgment. Uncertainty Architecture calls this a Thinking System. The name describes an engineering condition. It makes no claim about consciousness and does not imply that the product is well controlled or ready for production.

The relevant responsibility may be interpretation, classification, generation, routing or action selection. It is consequential when its result can materially affect an intended outcome, an applicable requirement, authority, resource use or someone downstream. That does not automatically mean high risk. Consequence, exposure and reversibility still need assessment.

Consider the talk's hypothetical internal IT assistant helping an employee recover access. It may explain approved guidance in different words, ask for missing context or escalate to support. Those variations can be useful. Inventing recovery steps, disclosing another employee's data or changing access without permission is a different matter.

Requirements therefore need to describe acceptable variation: approved sources, required meaning, examples, counterexamples, prohibited outcomes and the response to insufficient information. A prompt influences behavior; it does not specify every possible acceptable answer. Exact permissions, data boundaries and transaction rules remain engineering obligations. The team must identify how each important boundary is actually realized.

Evidence must fit the decision

A successful demonstration answers a narrow question: the candidate worked in that situation. Release requires evidence suited to the proposed use and consequences.

A curated reference set can probe ordinary, difficult and business-critical scenarios. It is valuable for regression and coverage. Its pass rate does not automatically estimate the error rate in production. Estimating a rate requires a justified sampling design, a defined population and attention to dependence, subgroup coverage and uncertainty. Repeating one scenario and testing many different scenarios answer different questions.

Evaluators also need evaluation. Developers, QA and domain experts should check which failures a scoring method misses, which useful answers it rejects and where human judgments disagree. Keep the model, prompt, retrieval, tools, rubric and test versions attached to the evidence. Behavior can change when these inputs change even if application code stays the same.

An average score can conceal a prohibited outcome. A good aggregate result does not authorize disclosure, and finding no disclosure in a test does not prove it impossible. Severity and frequency are separate parts of the decision.

Readiness, completion and release also answer different questions. Before work, agree the behavior, evidence plan, control path and resource limits. At completion, verify the implementation, evaluation and operational handover. The authorized release owner then decides whether this deployment and its residual risk are acceptable within the approved scope. A green run supplies evidence; it does not supply new authority.

Control must work before and after consequence

For the access-recovery assistant, checking whether an answer fits semantic criteria differs from checking whether the system may disclose information or execute an action. A semantic evaluator can be wrong. Permission checks and required approval must have a concrete enforcement path. Access boundaries also apply before data enters retrieval or model context; an output check cannot make unauthorized input access acceptable.

The example below separates these responsibilities and includes the rejected path. It illustrates one design, not a mandatory pipeline for every AI feature.

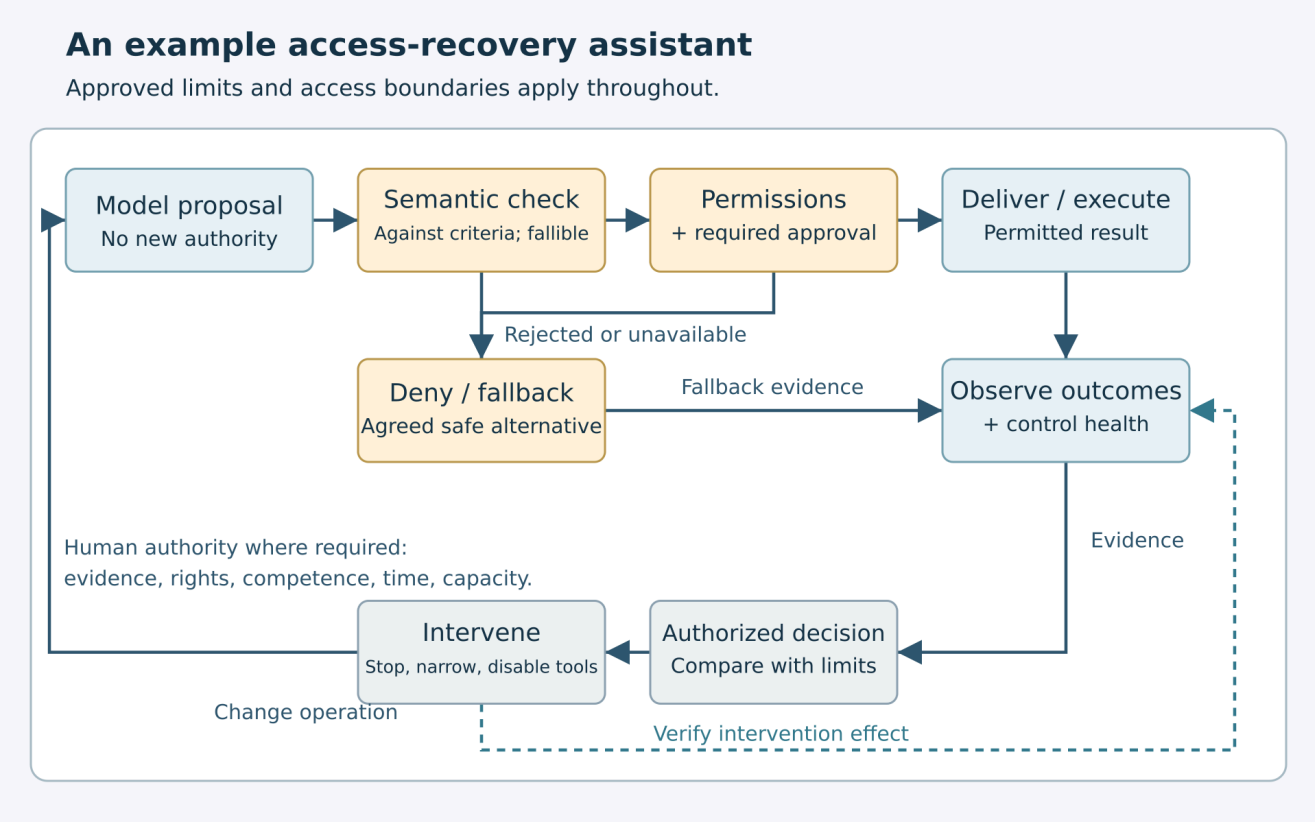


Figure 1. One schematic control path for the illustrative access-recovery assistant. Semantic checking is fallible and separate from permissions. Delivered, denied and fallback outcomes inform authorized intervention; a closed loop does not establish correctness. Original explanatory diagram for this article, based on PMDay v35 slide 12.

Observation should cover delivered results, denied requests, fallback outcomes and the health of the controls themselves. Someone or something with defined decision rights compares this evidence with approved limits. A real intervention can then stop or narrow operation, disable a tool or switch to an agreed fallback. Verify that the intervention changed operation as intended.

Timing matters. A response after an irreversible disclosure cannot undo it. Controls must act within the relevant consequence window. Where human approval or response is required, the person needs evidence, competence, authority, time, capacity and backup. A named reviewer who cannot respond under expected load is an unresolved dependency.

The full operating cost includes evaluation, retries, observation, human review and fallback, alongside the model call. Check whether the proposed controls and response capacity remain viable at the intended exposure. If a required boundary cannot be credibly realized, or effective intervention is too slow or unaffordable, narrow the use, redesign it or reject that AI path. Adding telemetry does not resolve those gaps.

Leadership connects the decisions

For enterprise leaders, the two changes expand what belongs in a delivery commitment. On the delivery side, review capacity, learning, ownership and recovery must keep pace with the accepted work. On the product side, evaluation, permissions, operational response, external dependencies and control cost become explicit parts of readiness.

Existing people can hold these responsibilities. Product and business owners define useful outcomes and approve applicable boundaries; analysts help make the criteria testable. Developers, QA and domain experts produce and interpret evidence. Architects and engineers implement controls. Release or risk owners make decisions within their authority, and service owners operate the accepted system. A project manager connects the required decisions, evidence and capabilities in the delivery plan without replacing technical or risk authority.

This coordination continues after launch. An implementation defect may need local correction. Increased tool authority, a changed provider assumption or a loss of fallback capacity may invalidate the project decision. A changed organizational boundary belongs with its owner. Reassessment follows the decision whose basis changed; a configurable system has no automatic permission to expand its scope.

Start with one workflow. State the useful outcome, approved limits and conditions for stopping. Identify the evidence needed for a decision, the people who can make it and the service owner who must respond. Rehearse the fallback before promising production capability. Then compare actual delivery and product outcomes with the agreed conditions and decide whether to continue, narrow or stop.

Further reading and the talk

The Subprime Code Crisis owns the delivery evidence and synthesis. Uncertainty Architecture organizes the runtime engineering responsibilities in a draft specification. Neither a framework name nor this explanation establishes that a particular control works.

This article follows the PMDay v35 source and notes, a 15-slide review candidate. Its editable deck and evidence inventory preserve the source versions and limitations. No empirical numbers are reproduced here.

PMDay 2026 · Recording in Ukrainian